



本小节内容

选择循环汇编实战解析

选择循环汇编实战解析

这一小节我们针对 C 语言的选择结构，循环结构，对应的汇编进行解析，首先我们编写下面 C 代码：

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i=5;
```

```
    int j=10;
```

```
    if (i < j)
```

```
    {
```

```
        printf("i is small\n");
```

```
    }
```

```
    for(i=0;i<5;i++)
```

```
    {
```

```
        printf("this is loop\n");
```

```
    }
```

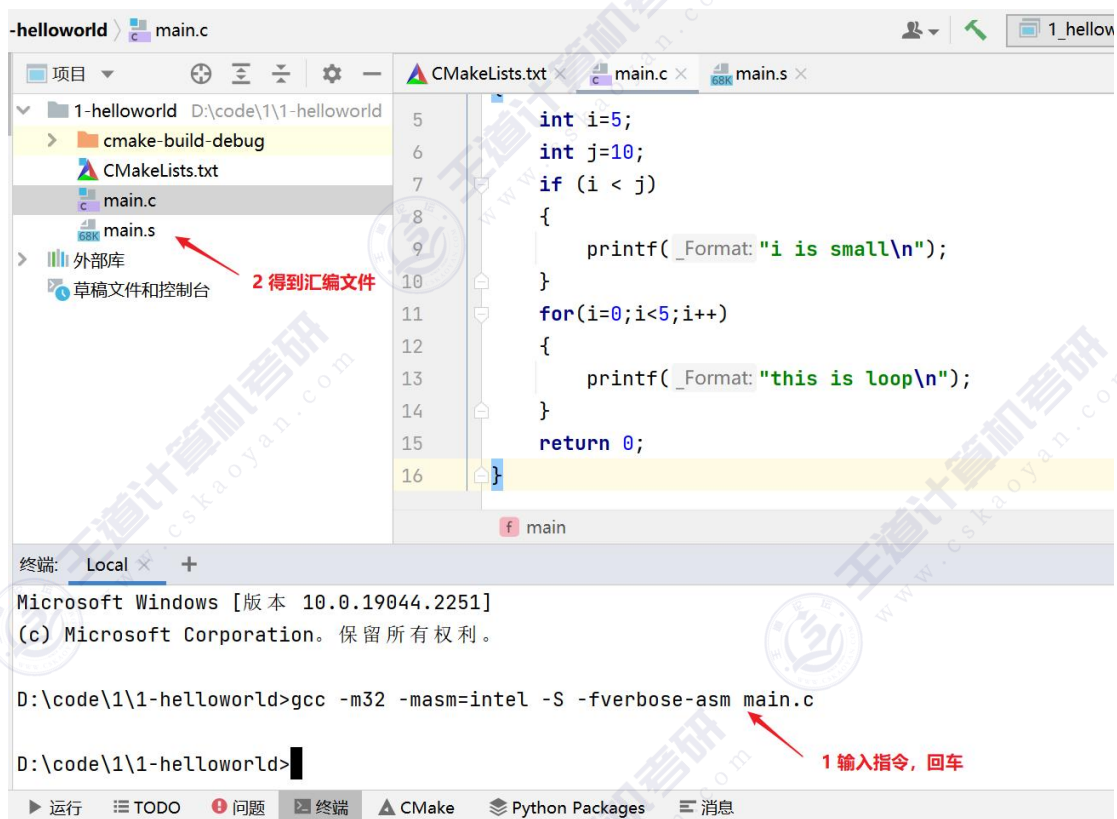
```
    return 0;
```

```
}
```

然后根据之前小节生成汇编的方法，在终端窗口输入



gcc -m32 -masm=intel -S -fverbose-asm main.c, 如下图所示, 得到汇编文件 main.s



接下来我们来分析转换后的汇编代码, 首先#号代表注释, 我们从 main 标签位置开看即可。我们的 C 代码在让 CPU 去运行时, 其实所有的变量名都已经消失了, 实际是数据从一个空间, 拿到另一个空间的过程。

栈内变量先定义在低地址还是高地址, 取决于操作系统与 CPU 的组合, 你的可能和我的不一样, 因此不用去研究, 没有意义 (也不属于考研大纲范围)。

我们访问所有变量的空间都是通过栈指针 (esp 时刻都存着栈指针, 也可以称为栈顶指针) 的偏移, 来获取对应变量内存空间的数据的。

.text#这里是文字常量区, 放了我们的字符串常量, LC0 和 LC1 分别是我们要用到的两个字符串常量的起始地址

```
.def __main; .scl 2; .type 32; .endef
```

```
.section .rdata,"dr"
```

LC0:

```
.ascii "i is small\0"
```

LC1:

```
.ascii "this is loop\0"
```



```
.text

.globl _main

.def    _main; .scl    2; .type    32; .endef

_main:

    push    ebp #

    mov ebp, esp #,

    and esp, -16 #,

    sub esp, 32 #,

    # main.c:4: {

    call    __main ##从 main 到这里这一段我们函数调用时解析，暂时不管

    # main.c:5:    int i=5;

    mov DWORD PTR [esp+28], 5 # i, #把常量 5 放入栈指针(esp 寄存器存的栈指针)偏移 28 个字节的位置，这个位置是变量 i 的空间

    # main.c:6:    int j=10;

    mov DWORD PTR [esp+24], 10 # j, #把常量 10 放入栈指针(esp 寄存器存的栈指针)偏移 24 个字节的位置，这个位置是变量 j 的空间

    # main.c:7:    if (i < j)

    mov eax, DWORD PTR [esp+28] # tmp89, i #把栈指针(esp 寄存器存的栈指针)偏移 28 个字节的位置内的值，放入 eax 寄存器

    cmp eax, DWORD PTR [esp+24] # tmp89, j #比较 eax 寄存器内的值和栈指针偏移 24 个字节位置的值的大小，拿 eax 寄存器值减去 DWORD PTR [esp+24]，然后设置条件码

    jge L2 #, #如果 eax 寄存器大于等于 DWORD PTR [esp+24]，那么跳转到 L2, 否则直接往下执行，jge 是根据条件码 ZF 和 SF 来判断的
```



```
# main.c:9:      printf("i is small\n");
```

```
mov DWORD PTR [esp], OFFSET FLAT:LC0    #, #把 LC0(也就是上面那个字符串)
```

的地址，放到寄存器栈指针指向的内存位置

```
call    _puts    #
```

L2:

```
# main.c:11:      for(i=0;i<5;i++)
```

```
mov DWORD PTR [esp+28], 0    # i, #把常量 0 放入栈指针(esp 寄存器存的栈指针)偏
```

移 28 个字节的位置，这个位置是变量 i 的空间

```
# main.c:11:      for(i=0;i<5;i++)
```

```
jmp L3    # #无条件跳转到 L3
```

L4:

```
# main.c:13:      printf("this is loop\n");
```

```
mov DWORD PTR [esp], OFFSET FLAT:LC1    #, #把 LC1(也就是上面那个字符串)
```

的地址，放到寄存器栈指针指向的内存位置

```
call    _puts    #
```

```
# main.c:11:      for(i=0;i<5;i++)
```

```
add DWORD PTR [esp+28], 1    # i,
```

L3:

```
# main.c:11:      for(i=0;i<5;i++)
```

```
cmp DWORD PTR [esp+28], 4    # i, #比较栈指针偏移 24 个字节位置的值与 4 的大小,
```

拿 DWORD PTR [esp+24]减去 4，然后设置条件码

```
jle L4    #, #小于等于就跳转到 L4
```

```
# main.c:15:    return 0;
```

```
    mov eax, 0    # _11,
```

```
# main.c:16: }
```

```
    leave
```

```
    ret
```

```
.ident "GCC: (x86_64-posix-sjlj-rev0, Built by MinGW-W64 project) 8.1.0"
```

```
.def    _puts; .scl    2; .type    32; .endef
```

这一节大家理解选择，循环的汇编指令及原理即可，主要掌握的指令是 `cmp`, `jge`, `jmp`, `jle` 等。以及了解一下字符串常量是存在文字常量区。